

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL X
Java Collections



Disusun Oleh:
Danu Dimas Putra 105224003

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Dalam permintaan THT, sistem simulasi manajemen gudang berfokus pada penerapan OOP (*Object-Oriented Programming*) dasar yang dalam penerapannya setiap atribut atau nilai perlu dienskapsulasi dan membutuhkan *method* seperti getter dan setter sebagai *treatment* dalam menangani nilai/data dari atribut dari objeknya. Dalam penerapan dasarnya juga membutuhkan sebuah *constructor* sebagai inisialisasi dari objek yang dibuat. Model objek yang diminta diawal berupa Barang dengan idBarang sebagai String, namaBarang dan kategori yang juga berupa String, lalu stok sebagai integer.

Selanjutnya *blueprint* berupa *class* SistemGudang juga perlu dirancang dengan penerapan tambahan yaitu *Collections* yang dijadikan atribut *class* dengan database sebagai utamanya memerlukan penggunaan yang menggunakan kunci berupa idBarang agar cepat dalam proses mencarinya, lalu kategori Unik yang menyimpan nilai/data dari objek barang setiap didaftarkan perlu memastikan tidak ada nama kategori ganda, lalu *collections* untuk riwayat yang menyimpan setiap kali ada catatan aktivitas pada SistemGudang. Terakhir memerlukan cetak laporan dari semua *collections* yang menyimpan nilai/*value* dibuat sebelumnya.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	idBarang	String	Untuk menyimpan huruf dari data id barang dan salah satu variabel yang berada sebagai atribut di dalam class.
2	namaBarang	String	Untuk menyimpan huruf dari data nama barang dan juga salah satu variabel yang berada sebagai atribut di dalam class.
3	kategori	String	Untuk menyimpan huruf dari data tipe kamar dan juga salah satu variabel yang berada sebagai atribut di dalam class.
4	stok	int	Untuk menyimpan nilai desimal dari data harga per malamnya dan salah satu variabel yang berada sebagai atribut di dalam class.
5	database	Map<String, Barang>	Variabel dengan Map yang menyimpan data dalam bentuk <i>Key-Value Pair</i> (Pasangan Kunci dan Nilai). Untuk sebuah kunci dari id barang dengan <i>value</i> objek barang tersebut.
6	kategoriUnik	Set<String>	Variabel dengan Set untuk menyimpan nilai yang jika sudah ada akan diabaikan (unik), variabel yang menola duplikat ketika menyimpan data kategori.
7	riwayat	List<String>	Variabel dengan ArrayList untuk menyimpan catatan aktivitas (riwayat) ketika terjadi penambahan stok/pengurangan.

8	barang	Barang	Sebagai variabel objek dari <i>class</i> Barang yang menyimpan pengembalian dari database yang mencari id objek ketika dibuat, lalu disimpan kembali ke variabel barang.
9	gudang	SistemGudang	Sebagai variabel objek dari <i>class</i> SistemGudang untuk menggunakan method atau perilaku tambah barang hingga mengurangi stok berdasarkan method yang menggunakan <i>java collections</i> .

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Barang()	<i>constructor</i>	Untuk inisialisasi setiap nilai atribut yang akan di-assign datanya saat objek dibuat.
2	getIdBarang()	<i>procedural</i>	Untuk pengembalian nilai atribut id barang yang akan memungkinkan kelas lain membaca nilai dari atribut private tanpa bisa mengubah nilainya secara langsung.
3	getNamaBarang()	<i>procedural</i>	Juga untuk pengembalian nilai atribut nama barang (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
4	getKategori()	<i>procedural</i>	Untuk pengembalian nilai atribut kategori barang (nilai huruf) yang berguna untuk baca saja tanpa bisa diubah nilainya.
5	getStok()	<i>procedural</i>	Untuk pengembalian nilai atribut stok barang (nilai angka) yang berguna untuk baca saja tanpa bisa diubah nilainya.
6	setStok()	<i>procedural</i>	Untuk dapat mengubah nilai dari stok dan membuat atribut jadi dapat di-set nilainya secara publik.
7	tambahBarangBaru()	<i>functional</i>	<i>Method</i> untuk menyimpan sebuah objek ke database (map) saat method dipanggil dan menjadikan nilai id barang menjadi sebuah kunci, kemudian nilai kategorinya disimpan ke dalam hashSet kategori unik agar setiap kategori yang baru dan unik disimpan ke dalam hashSet tersebut.
8	tambahStok()	<i>functional</i>	<i>Method</i> untuk menambah stok pada objek di dalam map database jika id dari barang ditemukan dan menyimpan aktivitasnya ke dalam sebuah riwayat (arrayList).
9	kurangiStok()	<i>functional</i>	<i>Method</i> untuk mengurangi stok ketika mencukupi berdasarkan kondisi jika id temukan lalu stok mencukupi.
10	cetakLaporan()	<i>functional</i>	<i>Method</i> untuk menampilkan/mencetak setiap nilai dari atribut dengan rapih dan detail.

IV. Dokumentasi dan Pembahasan Code

```
src > J Barang.java > Barang > setStok(int)
1  public class Barang {
2      private String idBarang;
3      private String namaBarang;
4      private String kategori;
5      private int stok;
6
7      Barang(String idBarang, String namaBarang, String kategori, int stok) {
8          this.idBarang = idBarang;
9          this.namaBarang = namaBarang;
10         this.kategori = kategori;
11         this.stok = stok;
12     }
13
14     public String getIdBarang() {
15         return idBarang;
16     }
17
18     public String getNamaBarang() {
19         return namaBarang;
20     }
21
22     public String getKategori() {
23         return kategori;
24     }
25
26     public int getStok() {
27         return stok;
28     }
29
30     public void setStok(int stok) {
31         this.stok = stok;
32     }
33 }
34
```

```

src > J SistemGudang.java > SistemGudang > cetakLaporan()
1  import java.util.ArrayList;
2  import java.util.HashMap;
3  import java.util.HashSet;
4  import java.util.List;
5  import java.util.Map;
6  import java.util.Set;
7
8  public class SistemGudang {
9      Map<String, Barang> database = new HashMap<>();
10     Set<String> kategoriUnik = new HashSet<>();
11     List<String> riwayat = new ArrayList<>();
12
13     public void tambahBarangBaru(String id, String nama, String kategori, int stok) {
14         database.put(id, new Barang(id, nama, kategori, stok));
15
16         kategoriUnik.add(kategori);
17     }
18
19     public void tambahStok(String id, int jumlah) {
20         Barang barang = database.get(id);
21
22         if (barang.getIdBarang() == id) {
23             barang.setStok(barang.getStok() + jumlah);
24             riwayat.add("Tambah Stok: " + id + " ditambah " + jumlah + ".");
25         }
26     }
27
28     public void kurangiStok(String id, int jumlah) {
29         Barang barang = database.get(id);
30
31         if (barang.getIdBarang() == id) {
32             if (barang.getStok() >= jumlah) {
33                 barang.setStok(barang.getStok() - jumlah);
34                 riwayat.add("Kurangi Stok: " + id + " dikurangi " + jumlah + ".");
35             }
36             else {
37                 System.out.println("Stok " + id + " tidak mencukupi, Sisa: " + barang.getStok() + "\n");
38             }
39         }
40     }
41
42     public void cetakLaporan() {
43         for (String daftarKategori : kategoriUnik) {
44             System.out.println("-" + daftarKategori);
45         }
46
47         System.out.println(x: "\n");
48
49         for (Barang barang : database.values()) {
50             System.out.println(barang.getNamaBarang() + " Sisa Stok : " + barang.getStok());
51         }
52
53         for (int i = 0; i < riwayat.size(); i++) {
54             System.out.println((i + 1) + ". " + riwayat.get(i));
55         }
56
57         System.out.println(x: "-----");
58     }
59 }
60

```

```

src > J Main.java
1  public class Main {
2      public static void main(String[] args) throws Exception {
3          SistemGudang gudang = new SistemGudang();
4
5          gudang.tambahBarangBaru(id: "B01", nama: "Laptop ACER RedBull Edition", kategori: "Elektronik", stok: 10);
6          gudang.tambahBarangBaru(id: "B02", nama: "Kursi Gaming Ferrari Pharaoh Edition", kategori: "Furnitur", stok: 5);
7          gudang.tambahBarangBaru(id: "B03", nama: "Baju RGB Galaxy UltraHD", kategori: "T-Shirt", stok: 5);
8
9          gudang.tambahStok(id: "B03", jumlah: 5);
10
11         gudang.kurangiStok(id: "B01", jumlah: 5);
12
13         gudang.kurangiStok(id: "B02", jumlah: 50);
14
15         gudang.cetakLaporan();
16     }
17 }
18

```

```

PS D:\Praktikum_PBO\praktikum_pbo-Modul10\praktikum_pbo-Mod10-THT> & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\Praktikum_PBO\praktikum_pbo-Mod10-THT\praktikum_pbo-Mod10-THT.jar' 'D:\Praktikum_PBO\praktikum_pbo-Mod10-THT\praktikum_pbo-Mod10-THT.jar'
Stok B02 tidak mencukupi, Sisa: 5

-Furnitur
-Elektronik
-T-Shirt

Laptop ACER RedBull Edition Sisa Stok : 5
Kursi Gaming Ferrari Pharaoh Edition Sisa Stok : 5
Baju RGB Galaxy UltraHD Sisa Stok : 10
1. Tambah Stok: B03 ditambah 5.
2. Kurangi Stok: B01 dikurangi 5.
-----
PS D:\Praktikum_PBO\praktikum_pbo-Modul10\praktikum_pbo-Mod10-THT>

```

A. Pembahasan Code

Pertama, dimulai dari setiap sintaks dalam *class* Barang yang menjadi bagian utama dalam pembuatannya. Deklarasikan setiap atribut variabel seperti `idBarang` berupa String, `namaBarang` berupa String, `kategori` berupa String, dan `stok` sebagai int (integer). Untuk konsep OOP (*Object-Oriented Programming*) dasar, penggunaan access modifier “private” diperlukan untuk menjaga atau mengenkapsulasi data atau nilainya dari *class-class* diluarnya. Melanjutkan konsep enkapsulasi, maka dibutuhkan *treatment* dengan getter sebagai pembaca nilainya dan setter yang dapat mengubah nilai dari atributnya secara public. Getter untuk setiap atributnya berfungsi untuk membaca dan mengambil nilainya, setter pada `stok` (`setStok()`) untuk mengubah seperti menambah/mengurangi nilainya nanti ketika dipanggil *method* diluar. *Constructor* `Barang()` setelah pembuatan atribut diperlukan untuk menginisialisasi setiap nilai atribut yang nantinya akan menjadi objek diluar *class*.

Selanjutnya dalam *class* `SistemGudang`, deklarasikan setiap collections sebagai atribut. Collections tersebut berfungsi sebagai tempat penyimpanan atau struktur data yang memiliki karakteristik tertentu, untuk atribut database yang berupa collections `Map` akan menyimpan nilainya disertai dengan kunci (*key*) untuk memanggil atau mengakses nilainya, untuk atribut `kategoriUnik` sebagai Set atau `HashSet` yang ketika menyimpan nilai yang sama akan diabaikan salah satunya karena nilai diharuskan unik dalam penggunaan Set, dan untuk atribut `riwayat` berupa List atau `ArrayList` yang dapat menyimpan secara terurut.

Method dalam *class* `SistemGudang`, untuk `tambahBarangBaru()` dimulai melakukan penyimpanan ke database yang telah dibuat dengan variabel `id` (`id` dari barang) sebagai kunci dan deklarasi objek langsung beserta nilai-nilainya sebagai *value* (nilai) di dalamnya. Kemudian melakukan penyimpanan ke `kategoriUnik` dengan variabel `kategori` yang juga menjadi nilai dari objek ketika dibuat sebelumnya. *Method* `tambahStok()` diawali dengan pengambilan *value* dari objek yang telah dibuat dengan memanggil kuncinya kemudian akan disimpan kembali ke variabel objek barang di *method* ini. Kemudian melakukan validasi, untuk sebuah barang yang sebelumnya

tersimpan dengan memanggil `.getIdBarang()`, lalu membandingkan variabel `id` yang dipanggil dan diisi diluar sama dengan objek `id` barang yang telah dibuat. Jika sama, maka objek barang ada dan dilakukan penambahan stok menggunakan `.setStok()` yang berisikan nilai stok dengan memanggil `getStok()` ditambah dengan jumlah yang akan diisikan, setelahnya dilakukan penambahan (simpan) ke Riwayat (List). *Method* `kurangiStok()` sama seperti sebelumnya namun dalam kondisinya menggunakan *nested if* untuk mengecek `id` dari objek barang tersedia kemudian stok yang ingin dikurangi apakah mencukupi. *Method* `cetakLaporan()` dimulai dengan melakukan perulangan untuk mencetak setiap nilai yang telah disimpan di variabel `Set` dari kategoriUnik dengan *foreach*, lanjut dengan perulangan selanjutnya yang akan mencetak `value` (nilai) dari variabel `Map` yang menyimpan objek barang sebelumnya dan disimpan kembali sebagai barang dari perulangan *foreach*, perulangan terakhir untuk mencetak setiap isi terurut yang disimpan dari variabel `List Riwayat`.

Terakhir dalam *class* `Main`, pertama deklarasi pembuatan objek gudang yang berdasarkan *class* `SistemGudang` sebagai sistem utamanya antara barang dengan user, kemudian untuk melakukan penambahan barang (objek barang) ke database `SistemGudang` maka dapat dipanggil *method* `tambahBarangBaru()` dengan disikan sebuah `id` sebagai kunci dan *value* yang berisi nilai-nilai dari sebuah barang (objek). Kemudian untuk menambahkan stok, panggil *method* `tambahStok()` kemudian diisikan kunci/`id` dari barang yang akan ditambah dan isikan jumlah stok yang ditambah. Untuk mengurangi sama seperti tambah sebelumnya dan untuk tes apakah validasinya berhasil, diberi jumlah yang diluar stok yang sudah diisikan sebelumnya. Kemudian dapat cetak laporannya dengan pemanggilan *method* `cetakLaporan()`.

V. Kesimpulan

Program simulasi sistem manajemen gudang ini, menghasilkan penerapan konsep OOP (*Object-Oriented Programming*) dasar yang mengenskapsulasi setiap nilai atribut dalam *class* berupa *access modifier* “private” yang penggunaannya hanya bisa digunakan di dalam *class* tersebut yang membuat setiap data dan nilainya aman dari perubahan secara langsung diluar *class* tersebut. Kemudian dalam melakukan perubahannya dibutuhkan metode tertentu seperti *getter* dan *setter*, *getter* sebagai pembaca dan yang mengembalikan atau mengambil nilai dari suatu atributnya dan *setter* yang dapat mengubah nilai dari atributnya dan membuatnya dapat diakses secara publik. Penggunaan *setter* yang diperuntukkan untuk stok saja karena dalam permintaan pembuatannya hanya berfokus ke penambahan dan pengurangan stok, jika perlu perubahan pada atribut lainnya, dapat ditambahkan *setter* untuk atribut tersebut dan dalam penerapan *setter*, dapat menggunakan validasi atau kondisi tertentu juga agar konsisten pada nilai dari atribut objek `Barang` tersebut.

Pembuatan *collections* dalam *class* `SistemGudang` yang dijadikan atribut juga berfungsi dalam penyimpanan struktur data yang menyimpan dengan karakteristik tertentu seperti, `Map` pada variabel `database` yang menyimpan setiap pembuatan objek barang disertai dengan kunci berupa `id` barang tersebut, kemudian `Set` pada variabel `kategoriUnik` yang menyimpan dengan mengabaikan nilai/data yang sama saat disimpan jadi seperti diharuskan unik, kemudian `List` atau `arrayList` pada variabel

riwayat yang penggunaannya untuk menyimpan secara terurut berdasarkan pemanggilan dan menyimpannya. Lalu pada Main, pembuatan objek dimulai dengan melalui SistemGudang kemudian pemanggilan *method* untuk penambahan objek barang serta menyimpan ke atribut database dari SistemGudang yang telah dibuat sebelumnya. Lalu tes untuk pemanggilan *method* tambahStok() dan kurangiStok() berdasarkan *method* yang telah dibuat sebelumnya hingga cetakLaporan() untuk melihat hasil dari menyimpan catatan aktivitas yang terjadi.

VI. Daftar Pustaka

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemograman Berorientasi Objek; Java Collections*. Universitas Pertamina.

GeeksforGeeks. (2026, 6 April). *Java Collection Tutorial*. GeeksforGeeks. <https://www.geeksforgeeks.org/java/java-collection-tutorial/>.